

Caching Strategies

Java Backend Interview — Short Notes

1. Why Cache? Caching Layers

- **Cache**: fast storage that holds a copy of data, reducing expensive re-computation or DB round-trips.
- **Goal**: reduce latency (ms → μs), reduce DB load, improve throughput, handle traffic spikes.

Layer	Location	Latency	Examples	Use Case
CPU Cache	L1/L2/L3 on processor	1–10 ns	Hardware managed	Transparent to developer
In-Process	JVM heap memory	Microseconds	Caffeine, Guava Cache, ConcurrentHashMap	Frequently read config, lookup tables, small datasets
Distributed	Separate cache server(s)	< 1 ms	Redis, Memcached, Hazelcast	Shared state across multiple app instances, sessions
CDN	Edge nodes near user	Milliseconds	CloudFront, Fastly, Akamai	Static assets, HTML, API responses for anonymous users
DB Query Cache	DB server memory	Microseconds	MySQL query cache (deprecated), PostgreSQL pg_cache	Repeated identical queries in same DB instance

2. Caching Strategies — Pros, Cons, Use Cases

Strategy	Flow	Pros	Cons	Best Use Case
Cache-Aside (Lazy Loading)	App: check cache → miss → read DB → populate cache → return.	Simple. Cache only what's needed. Resilient: DB still works if cache down.	Cache miss = 3 round trips. Thundering herd on cold start. Data can be slightly stale.	General-purpose. Read-heavy. Most common default choice. User profiles, product data.
Read-Through	App reads cache. On miss, cache automatically fetches from DB and stores.	App always talks to cache only. Consistent miss handling.	Cache provider must understand DB. Cold start misses. Added abstraction layer.	JPA L2 cache (Hibernate). When you want transparent caching in ORM.
Write-Through	App writes to cache → cache synchronously writes to DB → returns.	Cache always consistent with DB. No stale reads.	Write latency (2 writes). Cache stores data that may never be read.	Data that's written then immediately re-read. Payment status updates. User preferences.
Write-Behind (Write-Back)	App writes to cache → returns immediately → cache async writes to DB.	Lowest write latency. Batch writes to DB. Good for write spikes.	Data loss if cache crashes before DB write. Complex. Risk of stale DB.	High-write throughput: view counters, like counts, analytics events.
Write-Around	App writes directly to DB, bypassing cache. Reads still use cache.	Cache not polluted with write-once data.	First read after write is always a cache miss.	Write-rarely-read-rarely data. Log entries. Archive records.

Refresh-Ahead (Pre-fetch)	Cache proactively refreshes entries before they expire (based on access patterns).	Low latency — cache rarely expires during active use.	Wasteful if prediction wrong. Complex to implement.	Highly predictable access: top-N product pages, daily report data, exchange rates.
----------------------------------	--	---	---	--

3. Cache Invalidation — The Hard Problem

- "There are only two hard things in CS: cache invalidation and naming things." — Phil Karlton

3.1 Invalidation Strategies

Strategy	How	Pros	Cons	Example
TTL (Time To Live)	Key auto-expires after N seconds.	Simple, automatic, no extra logic.	Stale data until TTL expires. Unknown how stale.	Product price valid for 5 minutes. <code>setex product:123 300 '...'</code>
Event-driven invalidation	On DB write, immediately delete or update cache key.	Near-zero staleness. Precise control.	Dual-write consistency challenge: what if cache delete fails?	On order status update, <code>DELETE cache:order:{id}</code>
Version-based	Embed version in cache key. Increment version on change.	Old and new data coexist. Blue/green cache.	Old keys accumulate — need TTL cleanup.	<code>user:v5:profile:123</code> — change to <code>user:v6:profile:123</code>
Cache stampede prevention	Probabilistic early expiry: random refresh before TTL. Or lock on miss.	Prevents thundering herd on expiry.	Complexity. Still possible brief stale data.	Use mutex: first miss acquires lock, refreshes, others wait.
Write-through invalidation	Update cache on every write.	Always consistent.	Slow writes. Cache grows with every DB write.	User settings — always write to cache + DB together.

3.2 Cache Stampede / Thundering Herd

- Many concurrent requests for same expired/missing key → all miss → all hit DB simultaneously → DB overloaded.
- **Solution 1 — Mutex lock:** first thread to miss acquires distributed lock, fetches, populates. Others wait.
- **Solution 2 — Stale-while-revalidate:** serve stale data while background thread refreshes.
- **Solution 3 — Probabilistic early expiry:** refresh key slightly before TTL with probability proportional to staleness.
- **Solution 4 — Background refresh scheduler:** scheduled job refreshes hot keys before they expire.

```
// Redis mutex lock to prevent stampede
public Product getProduct(Long id) {
    String cached = redis.get("product:" + id);
    if (cached != null) return deserialize(cached);

    // Miss – acquire lock
    String lockKey = "lock:product:" + id;
    Boolean acquired = redis.setIfAbsent(lockKey, "1", 5, SECONDS);
    if (acquired) {
        try {
            Product p = db.findById(id);
            redis.setex("product:" + id, 300, serialize(p));
            return p;
        } finally { redis.delete(lockKey); }
    } else {
        Thread.sleep(50); return getProduct(id); // retry
    }
}
```

}

4. Cache Problems & Solutions

Problem	Description	Solution
Cache Penetration	Query for key that doesn't exist in DB or cache. Every request hits DB.	Bloom filter: check if key could exist before DB query. Or cache null/empty result with short TTL.
Cache Breakdown	Hot key expires. Massive concurrent requests all miss and hit DB simultaneously.	Never expire hot keys + background refresh. Or mutex lock on miss.
Cache Avalanche	Many keys expire at same time → mass DB flood.	Randomize TTL: <code>base_ttl + random(0..300s)</code> . Stagger expiry times.
Cache Pollution	Low-use data fills cache, evicting high-use data.	LRU/LFU eviction policies. Separate caches for hot vs cold data. Partial caching.
Stale Data	Cache has old data after DB update.	Short TTL for fast-changing data. Event-driven invalidation. Write-through strategy.
Cache Inconsistency	DB updated but cache delete failed → perpetually stale.	Retry cache invalidation. Use transactions where possible. Accept eventual consistency + short TTL.

5. Spring Boot Caching

```
// Enable caching
@EnableCaching // on @SpringBootApplication or @Configuration

// Cache a method result
@Cacheable(value = "products", key = "#id",
condition = "#id != null",
unless = "#result == null")
public Product getProduct(Long id) {
    return productRepo.findById(id).orElse(null);
}

// Evict on update
@CacheEvict(value = "products", key = "#product.id")
public Product updateProduct(Product product) { ... }

// Update cache entry on save
@CachePut(value = "products", key = "#result.id")
public Product saveProduct(Product product) { ... }

// Evict all entries in a cache
@CacheEvict(value = "products", allEntries = true)
public void clearProductCache() { }

# Redis as Spring Cache backend
spring.cache.type=redis
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.cache.redis.time-to-live=300000 # 5 minutes in ms
```

6. Interview Questions

Strategies & Use Cases

Q1. Cache-aside vs write-through — when to use each?

→ Cache-aside: lazy loading, only cache what's read, simple, most common default. Use when reads far exceed writes and cache misses are acceptable. Write-through: always consistent, zero stale risk. Use when data is written then immediately read (user preferences, payment status). Write-through wastes cache space on write-heavy, rarely-read data.

Q2. How do you decide what to cache and for how long?

→ Cache: frequently read, expensive to compute/fetch, relatively stable data. Don't cache: high-churn data (real-time stock price), personalised data (different per user unless keyed), security-sensitive data (permissions). TTL: fast-changing = short TTL (30s); stable reference data = long TTL (1hr+); session data = match session timeout.

Q3. How would you cache a list/collection result?

→ Option 1: cache entire list under one key — fast but invalidation invalidates everything. Option 2: cache individual items + a separate index list. Option 3: cache paginated results under page-specific key. Prefer individual item caching for write-heavy collections, list caching for mostly-static reference data.

Q4. Describe a realistic caching architecture for an e-commerce site.

→ CDN: static assets + product images. Redis L1: product catalog (TTL 5min), inventory counts (write-through), user session, shopping cart. DB query cache: complex product listing queries materialized. Caffeine in-process: exchange rates, config, country codes (TTL 1hr, tiny datasets). Bloom filter: prevent cache penetration for invalid product IDs.

Invalidation & Problems

Q5. What is cache stampede and how do you prevent it?

→ When a popular cached key expires, all concurrent requests miss, flood the DB simultaneously. Prevention: (1) Mutex lock — first miss fetches, others wait. (2) Probabilistic early refresh — randomly refresh before expiry. (3) Background refresh — scheduled job refreshes hot keys before they expire. (4) Stale-while-revalidate — serve old data while refreshing asynchronously.

Q6. How do you keep cache and DB consistent in a distributed system?

→ Ideal: write-through (synchronous) or CDC (Debezium reads DB transaction log, invalidates cache). Practical: event-driven invalidation on write + short TTL fallback. Accept eventual consistency for most data. Use versioned keys for blue/green deploys. Critical financial data: skip cache, read from DB directly.

Q7. What eviction policy would you use for a product recommendation cache?

→ LRU (Least Recently Used) if access pattern is recency-based — most recently viewed recommendations stay fresh. LFU (Least Frequently Used) if some products are always popular — evict rarely accessed items. For Redis: allkeys-lru for general cache (evict any key). volatile-lru if mixing persistent and cache keys.